МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И УПРАВЛЯЮЩИХ СИСТЕМ

Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

Лабораторная работа №2

по дисциплине: «Тестирование ПС»

по теме: «Интеграционное и API-тестирование»

Выполнил: студент группы ПВ-233

Алексеенко Андрей Евгеньевич

Проверил: преподаватель

Доденко Олег Павлович

Белгород, 2026 г.

Цель работы: освоить тестирование REST API на примере открытого публичного сервиса: научиться строить в Postman коллекцию запросов, писать автоматические проверки и анализировать расхождения между документацией и фактическим поведением API.

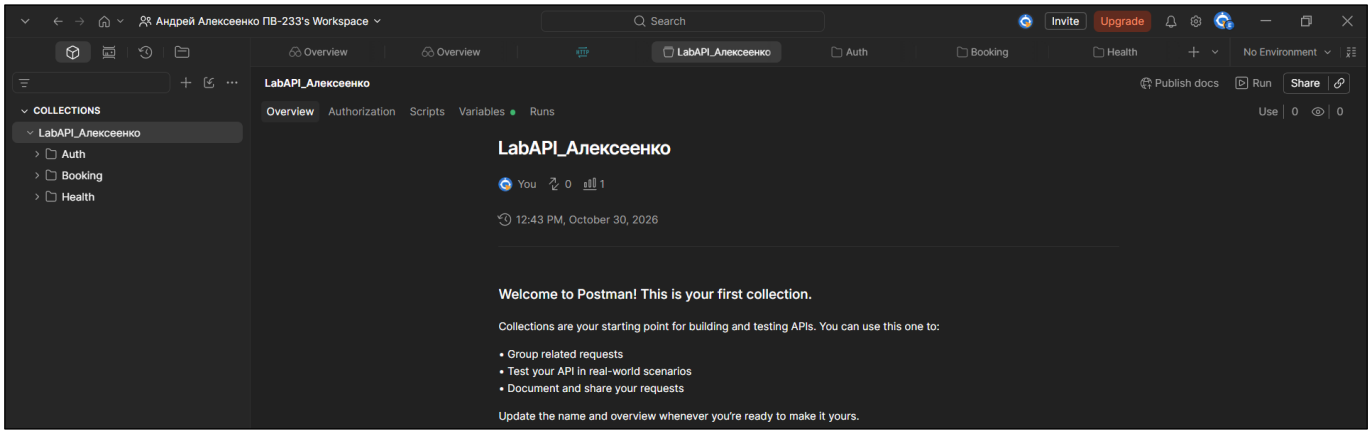
Содержание

Задание 1 3
Задание 2 3
Задание 3 4
Задание 4 5
Задание 5 6
Задание 6 8
Задание 7 8
Вывод 10

Задания

1. Создайте в Postman коллекцию LabAPI_<Фамилия> с переменными окружения (baseUrl, token, bookingId) и папками по ресурсам: Auth, Booking, Health.
2. Добавьте запросы ко всем эндпоинтам из таблицы. Для каждого запроса напишите описание: цель, входные данные, ожидаемый результат.
3. Автоматизируйте получение токена. В запросе POST /auth скриптом во вкладке Tests сохраните токен в переменную окружения. В запросе создания бронирования сохраните bookingid.
4. Составьте и выполните тест-кейсы в каждой категории: позитивные, негативные, граничные значения, коды ответов.
5. Для каждого запроса добавьте во вкладку Tests минимум 3 автоматические проверки.
6. Запустите всю коллекцию через Collection Runner и убедитесь, что цепочка сценариев работает без ручного вмешательства (токен и id передаются через переменные).
7. Фактическое и ожидаемое поведение. Это публичное учебное API, и его реальное поведение может отличаться от документации и от «правильного» REST (нестандартные коды для ошибок, код ответа для DELETE и т. д.). Оформите таблицу расхождений.

Задание 1. Создайте в Postman коллекцию LabAPI_<Фамилия> с переменными окружения (baseUrl, token, bookingId) и папками по ресурсам: Auth, Booking, Health.

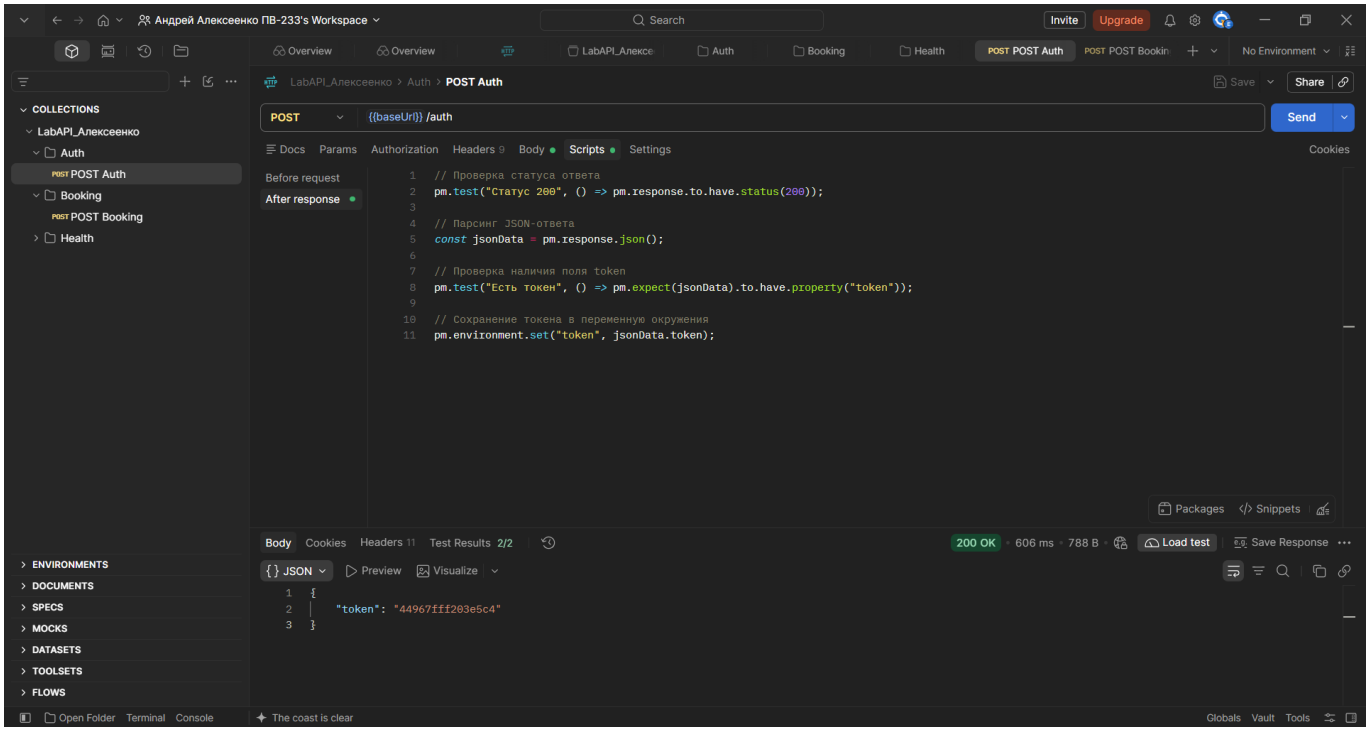


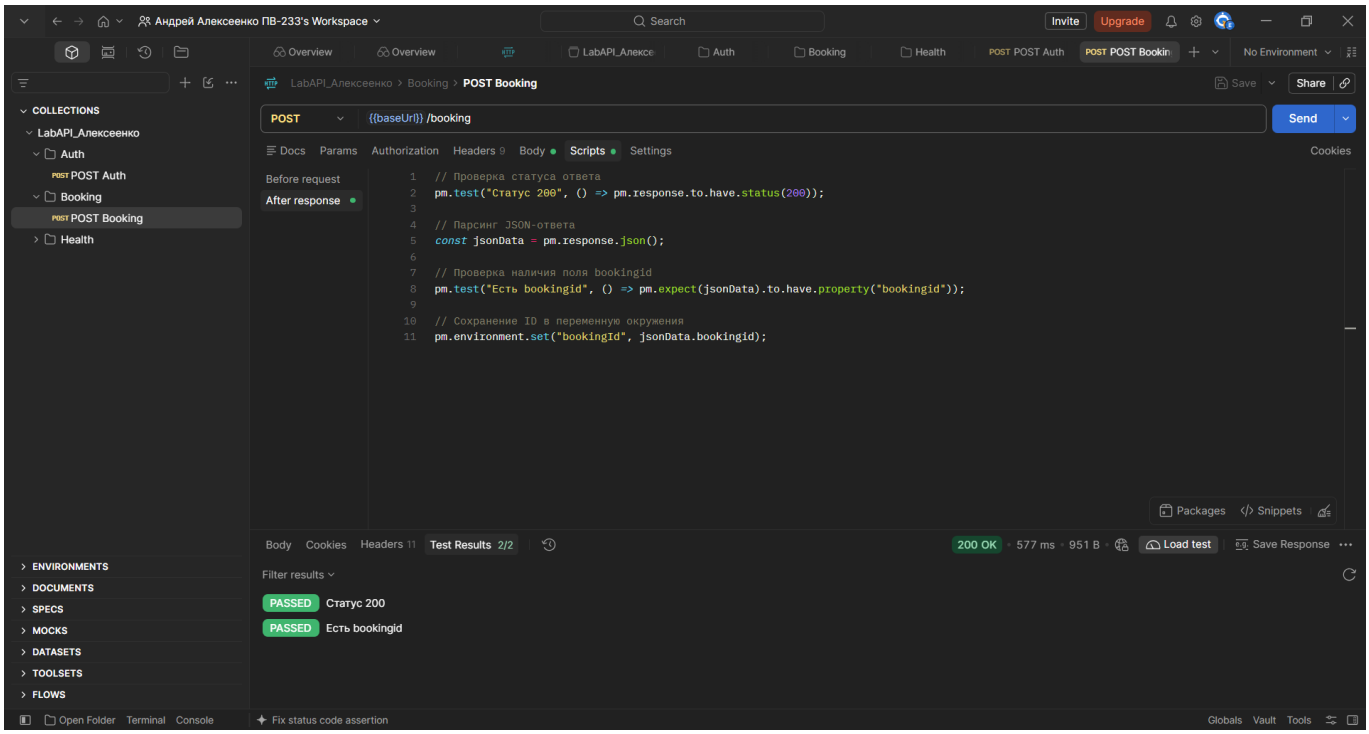
Задание 2. Добавьте запросы ко всем эндпоинтам из таблицы. Для каждого запроса напишите описание: цель, входные данные, ожидаемый результат.

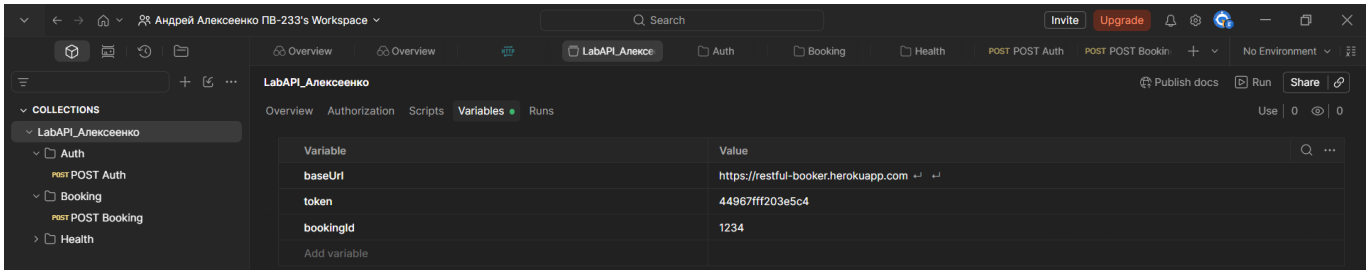
Папка	Запрос (Метод + URL)	Цель	Входные данные	Результат
Auth	POST {{baseUrl}}/auth	Получение токена для доступа к защищённым методам.	Body (JSON): {"username": "admin", "password": "password123"}	Код 200 OK. В ответе есть поле token. Скрипт сохраняет токен в переменную.
Booking	POST {{baseUrl}}/booking	Создание нового бронирования и получение его ID.	Body (JSON): firstname, lastname, totalprice, depositpaid, bookingdates, additionalneeds.	Код 200 OK. В ответе есть bookingid. Скрипт сохраняет ID в переменную.
Booking	GET {{baseUrl}}/booking/{{bookingId}}	Получение информации о созданном бронировании.	Переменная пути: bookingId.	Код 200 OK. Тело ответа содержит данные, отправленные при создании.
Booking	PUT {{baseUrl}}/booking/{{bookingId}}	Полное обновление (замена) всех полей бронирования.	Headers: Cookie: token={{token}}. Body (JSON): полный набор полей с измененным firstname: "James".	Код 200 OK. В ответе возвращаются обновлённые данные.
Booking	PATCH {{baseUrl}}/booking/{{bookingId}}	Частичное обновление (изменение	Headers: Cookie: token={{token}}. Body	Код 200 OK. В ответе имя изменено,

		только одного поля).	(JSON): {"firstname": "James"}	остальные поля нетронуты.
Booking	DELETE {{baseUrl}}/booking/{{bookingId}}	Удаление бронирования из системы.	Headers: Cookie: token={{token}}.	Код 201 Created *(особенность API, вместо ожидаемого 200/204)*.
Booking	DELETE {{baseUrl}}/booking/1	Негативный тест: проверка запрета на удаление без авторизации.	Headers: отсутствуют (нет токена).	Код 403 Forbidden (доступ запрещён).
Health	GET {{baseUrl}}/ping	Проверка работоспособности API (health-check).	Отсутствуют.	Код 201 Created *(особенность API)*. Время ответа < 2000 мс.

Задание 3. Автоматизируйте получение токена. В запросе POST /auth скриптом во вкладке Tests сохраните токен в переменную окружения. В запросе создания бронирования сохраните bookingid.







Задание 4. Составьте и выполните тест-кейсы в каждой категории.

ID	Эндпоинт	Тип	Входные данные	Ожидаемый код	Фактический	Статус
TC-01	POST /booking	Позитивный	Валидный JSON (создание)	200	200	Passed
TC-02	GET /booking/{id}	Позитивный	Существующий bookingId	200	200	Passed
TC-03	PUT /booking/{id}	Позитивный	Токен + валидный JSON	200	200	Passed
TC-04	PATCH /booking/{id}	Позитивный	Токен + {"firstname": "James"}	200	200	Passed
TC-05	DELETE /booking/{id}	Позитивный	Токен + существующий ID	201	201	Passed
TC-06	GET /booking/{id}	Негативный	ID, который только что удалили	404	404	Passed
TC-07	POST /auth	Негативный	Неверный пароль (wrongpass)	200	200	Passed

TC-08	DELETE /booking/1	Негативный	Без токена в Headers	403	403	Passed
TC-09	PUT /booking/1	Негативный	Неверный токен (Cookie: token=123)	403	403	Passed
TC-10	GET /booking/999999	Негативный	Несуществующий ID	404	404	Passed
TC-11	POST /booking	Негативный	Синтаксическая ошибка в JSON (пропущена кавычка)	400	500	Failed
TC-12	POST /booking	Негативный	"totalprice": "сто" (строка вместо числа)	400	500	Failed
TC-13	POST /booking	Негативный	Отсутствует обязательное поле firstname	400	500	Failed
TC-14	POST /booking	Граничный	"firstname": "" (пустая строка)	400/200	200	Passed (особенность)
TC-15	POST /booking	Граничный	"firstname": "A"*1000 (очень длинная строка)	400/200	200	Passed (особенность)
TC-16	POST /booking	Граничный	"totalprice": 0	400/200	200	Passed (особенность)
TC-17	POST /booking	Граничный	"totalprice": -100 (отрицательное)	400/200	200	Passed (особенность)
TC-18	POST /booking	Граничный	"totalprice": 999999999 (огромное)	400/200	200	Passed (особенность)
TC-19	POST /booking	Граничный	checkout раньше checkin	400	200	Passed (особенность)
TC-20	POST /booking	Граничный	checkin: "2024-13-45" (невалидная дата)	400	500	Failed

Задание 5. Для каждого запроса добавьте во вкладку Tests минимум 3 автоматические проверки.

1. POST Auth (Авторизация)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000мс", () => pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Есть токен", () => pm.expect(pm.response.json()).to.have.property("token"));
pm.environment.set("token", pm.response.json().token);
```

2. POST Booking (Создание)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000мс", () => pm.expect(pm.response.responseTime).to.be.below(2000));
```

```
pm.test("Есть bookingid", () => pm.expect(pm.response.json()).to.have.property("bookingid"));
pm.test("Данные сохранились верно", () =>
pm.expect(pm.response.json().booking.firstname).to.eql("Jim"));
pm.environment.set("bookingId", pm.response.json().bookingid);
```

3. GET Booking (Получение)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000мс", () => pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Имя совпадает", () => pm.expect(pm.response.json().firstname).to.eql("Jim"));
pm.test("Фамилия совпадает", () => pm.expect(pm.response.json().lastname).to.eql("Brown"));
```

4. PUT Booking (Полное обновление)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000мс", () => pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Имя обновилось на James", () =>
pm.expect(pm.response.json().firstname).to.eql("James"));
pm.test("Фамилия не изменилась", () =>
pm.expect(pm.response.json().lastname).to.eql("Brown"));
```

5. PATCH Booking (Частичное обновление)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000мс", () => pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Поле firstname изменено", () =>
pm.expect(pm.response.json().firstname).to.eql("James"));
pm.test("Остальные поля не тронуты", () =>
pm.expect(pm.response.json().lastname).to.eql("Brown"));
```

6. DELETE Booking (Удаление)

```
pm.test("Статус 201 Created", () => pm.response.to.have.status(201));
pm.test("Время ответа < 2000мс", () => pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Тело ответа пустое или Created", () =>
pm.expect(pm.response.text()).to.include("Created"));
```

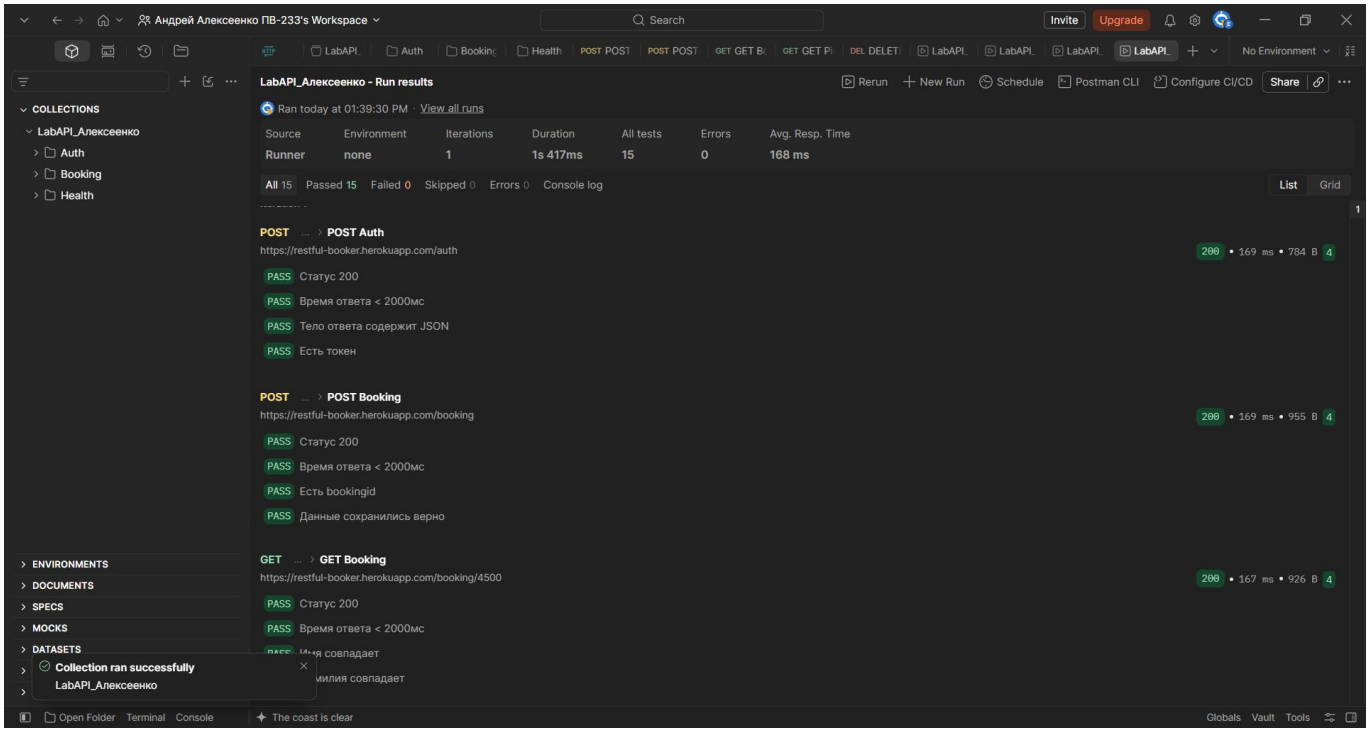
7. GET Ping (Health Check)

```
pm.test("Статус 201 Created", () => pm.response.to.have.status(201));
pm.test("Время ответа < 2000мс", () => pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Тело ответа содержит Created", () =>
pm.expect(pm.response.text()).to.include("Created"));
```

8. DELETE No Auth (Негативный тест: удаление без токена)

```
pm.test("Статус 403 Forbidden", () => pm.response.to.have.status(403));
pm.test("Время ответа < 2000мс", () => pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Тело ответа содержит Forbidden", () =>
pm.expect(pm.response.text()).to.include("Forbidden"));
```

Задание 6. Запустите всю коллекцию через Collection Runner и убедитесь, что цепочка сценариев работает без ручного вмешательства (токен и id передаются через переменные).



Задание 7. Фактическое и ожидаемое поведение. Это публичное учебное API, и его реальное поведение может отличаться от документации и от «правильного» REST (нестандартные коды для ошибок, код ответа для DELETE и т. д.). Оформите таблицу расхождений.

№	Запрос	Ожидалось	Получено	Вывод
1	DELETE /booking/{id}	200 OK или 204 No Content	201 Created	Особенность API. Сервер возвращает код успешного создания, хотя объект удаляется.
2	POST /booking (невалидный JSON)	400 Bad Request (с указанием ошибки)	500 Internal Server Error	Дефект. Ошибка валидации клиента приводит к падению сервера (500).
3	POST /booking (отсутствует firstname)	400 Bad Request	500 Internal Server Error	Дефект. Нет проверки обязательных полей, сервер падает при попытке их обработать.
4	POST /booking (totalprice = строка)	400 Bad Request	500 Internal Server Error	Дефект. Сервер не проверяет типы данных и падает из-

				за неверного формата.
5	POST /booking (checkin = "2024-13-45")	400 Bad Request	500 Internal Server Error	Дефект. Невалидная дата не отлавливается валидацией, вызывая ошибку 500.
6	POST /booking (totalprice = -100)	400 Bad Request	200 OK	Особенность/Дефект. API принимает отрицательную цену, нарушая бизнес-логику.
7	POST /booking (firstname = "")	400 Bad Request	200 OK	Особенность. API принимает пустые строки как валидные данные.
8	POST /booking (длинная строка 1000+)	400 Bad Request (если есть лимит)	200 OK	Особенность. Сервер не ограничивает длину строк.
9	POST /booking (checkout раньше checkin)	400 Bad Request (ошибка логики)	200 OK	Особенность/Дефект. API не проверяет хронологию дат.
10	POST /auth (неверный пароль)	401 Unauthorized	200 OK (с телом "reason": "Bad credentials")	Особенность. API возвращает 200 при ошибке авторизации, а не 401.
11	PUT/PATCH/DELETE (без токена)	401 Unauthorized	403 Forbidden	Особенность. Вместо «Не авторизован» сервер отвечает «Доступ запрещен».
12	PUT/PATCH/DELETE (неверный токен)	401 Unauthorized	403 Forbidden	Особенность. То же самое: невалидный токен приравнивается к отсутствию прав.
13	GET /booking/{id} (удаленный)	404 Not Found	404 Not Found	Соответствует REST. (Можно не писать, если препод требует только расхождения).

Баг-репорт 1 (Серьёзность: Major)

- Шаги: отправить POST /booking с телом {"firstname": "Jim", "totalprice": "сто"}.
- Ожидаемый результат: код 400 Bad Request с указанием ошибки валидации.
- Фактический результат: код 500 Internal Server Error.

Баг-репорт 2 (Серьёзность: Major)

- Шаги: отправить POST /booking без обязательного поля firstname.
- Ожидаемый результат: код 400 Bad Request.
- Фактический результат: код 500 Internal Server Error.

Баг-репорт 3 (Серьёзность: Minor)

- Шаги: отправить POST /booking с totalprice = -100.
- Ожидаемый результат: код 400 Bad Request (цена не может быть отрицательной).
- Фактический результат: код 200 OK (бронь создаётся).

Вывод: выполняя лабораторную работу, я освоил интеграционное и API-тестирование в Postman: создал коллекцию с автоматическим сохранением токена и ID в переменные, написал скрипты проверок и выявил ряд отклонений публичного API от стандартов REST и документации, что подтвердило важность тестирования для контроля фактического поведения системы.